

Question 1 — IoT Sensor Dashboard

Problem 1 — Sensor Reading Classifier

```
#include <iostream>
using namespace std;

int main() {

    double c;

    cout << "Enter Reading : ";
    cin >> c;

    int status_code;
    string status_label;
    string action;

    if (c < 0) {
        status_code = -1;
    }
    else if ((c >= 0) && (c <= 29)) {
        status_code = 0;
    }
    else if ((c >= 30) && (c <= 44)) {
        status_code = 1;
    }
    else if ((c >= 45) && (c <= 59)) {
        status_code = 2;
    }
    else {
        status_code = 3;
    }

    switch (status_code) {

        case -1: {
            status_label = "SENSOR_ERROR";
            action = "Sensor fault — check wiring";
            break;
        }

        case 0: {
```

```

        status_label = "NORMAL";
        action = "No action required";
        break;
    }

    case 1: {
        status_label = "WARNING";
        action = "Alert sent to supervisor";
        break;
    }

    case 2: {
        status_label = "CRITICAL";
        action = "Cooling system triggered";
        break;
    }

    case 3: {
        status_label = "SHUTDOWN";
        action = "Emergency shutdown initiated";
        break;
    }
}

double f = (c * 9 / 5) + 32;

cout << "Temperature   : " << c << "°C / " << f << "°F" << endl;

cout << "Status       : " << status_label << endl;
cout << "Action       : " << action << endl;

string reading = (c >= 25) ? "Above Average" : "Below Average";

cout << "Reading      : " << reading;

return 0;
}

```

```
PS D:\C++\Assignment\Assignment 01> cd "d:\C++\Assignment\Assignment 01\"
Enter Reading : 47.3
Temperature    : 47.3T C / 117.14T F
Status        : CRITICAL
Action        : Cooling system triggered
Reading       : Above Average
PS D:\C++\Assignment\Assignment 01> █
```

Problem 2 — Sensor Log Buffer

```
#include <iostream>
using namespace std;

int main() {

    int n;

    cout << "Enter Number of readings : ";
    cin >> n;

    if (n < 1 || n > 100) {
        cout << "Invalid number of readings";
        return 1;
    }

    float a[100];

    for (int i = 0; i < n; i++) {
        cout << "Enter reading : ";
        cin >> a[i];
    }

    cout << "Readings entered : " << n << endl;

    cout << "Valid readings : ";

    int skipped = 0;

    for (int i = 0; i < n; i++) {

        if (a[i] < 0.0) {
            skipped++;
            continue;
        }
    }
}
```

```
}

cout << a[i] << " ";
}

cout << endl;

cout << "Skipped (errors) : " << skipped << endl;

for (int i = 0; i < n; i++) {

    if (a[i] >= 45.0) {
        cout << "First CRITICAL : Index " << i
            << " → " << a[i] << "°C" << endl;
        break;
    }
}

float min = a[0];
float max = a[0];
float sum = 0.0;
int validCount = 0;

for (int i = 0; i < n; i++) {

    if (a[i] >= 0.0) {

        if (validCount == 0) {
            min = a[i];
            max = a[i];
        }

        if (a[i] < min) {
            min = a[i];
        }

        if (a[i] > max) {
            max = a[i];
        }

        sum = sum + a[i];
        validCount++;
    }
}
```

```
float avg = sum / validCount;
```

```
cout << "Min : " << min << "°C ";
```

```
cout << "Max : " << max << "°C ";
```

```
cout << "Avg : " << avg << "°C" << endl;
```

```
int cN = 0;
```

```
int cW = 0;
```

```
int cC = 0;
```

```
int cS = 0;
```

```
for (int i = 0; i < n; i++) {
```

```
    if ((a[i] >= 0.0) && (a[i] <= 29.0)) {
```

```
        cN++;
```

```
    }
```

```
    else if ((a[i] >= 30.0) && (a[i] <= 44.0)) {
```

```
        cW++;
```

```
    }
```

```
    else if ((a[i] >= 45.0) && (a[i] <= 59.0)) {
```

```
        cC++;
```

```
    }
```

```
    else if (a[i] >= 60.0) {
```

```
        cS++;
```

```
    }
```

```
}
```

```
cout << "Normal:" << cN << " ";
```

```
cout << "Warning:" << cW << " ";
```

```
cout << "Critical:" << cC << " ";
```

```
cout << "Shutdown:" << cS;
```

```
return 0;
```

```
}
```

```

PS D:\C++\Assignment\Assignment 01> cd "d:\C++\Assignment\Assignment 01\"
Enter Number of readings : 8
Enter reading : 22.1
Enter reading : 31.5
Enter reading : -5
Enter reading : 46
Enter reading : 28
Enter reading : 50.2
Enter reading : 10
Enter reading : 38
Readings entered : 8
Valid readings : 22.1 31.5 46 28 50.2 10 38
Skipped (errors) : 1
First CRITICAL : Index 3 46°C
Min : 10°C Max : 50.2°C Avg : 32.2571°C
Normal:3 Warning:2 Critical:2 Shutdown:0
PS D:\C++\Assignment\Assignment 01>

```

Problem 3 — Building Sensor Grid

```

#include <iostream>
#include <iomanip>
using namespace std;

int main() {

    double arr[3][3];
    int r = 3;
    int c = 3;

    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) {
            cout << "Floor : " << i + 1
                << " Room : " << j + 1 << " : ";
            cin >> arr[i][j];
        }
        cout << endl;
    }

    cout << fixed << setprecision(1);

    cout << setw(12) << "Room 1"
        << setw(12) << "Room 2"
        << setw(12) << "Room 3" << endl;
}

```

```

for (int i = 0; i < r; i++) {

    cout << "Floor " << i + 1 << " : ";

    for (int j = 0; j < c; j++) {
        cout << setw(7) << arr[i][j];
    }

    cout << endl;
}

double max = arr[0][0];

int index1 = 0;
int index2 = 0;

for (int i = 0; i < r; i++) {
    for (int j = 0; j < c; j++) {

        if (arr[i][j] > max) {
            max = arr[i][j];
            index1 = i;
            index2 = j;
        }
    }
}

cout << "Hottest Room : Floor " << index1 + 1
    << ", Room " << index2 + 1
    << " -> " << max << " C" << endl;

double highestAvg = 0.0;
int hottestFloor = 0;

for (int i = 0; i < r; i++) {

    double sum = 0.0;

    for (int j = 0; j < c; j++) {
        sum = sum + arr[i][j];
    }

    double avg = sum / 3.0;

```

```
    if (i == 0 || avg > highestAvg) {  
        highestAvg = avg;  
        hottestFloor = i;  
    }  
}  
  
cout << fixed << setprecision(2);  
  
cout << "Hottest Floor : Floor " << hottestFloor + 1  
    << " (avg " << highestAvg << " C)" << endl;  
  
int counter = 0;  
  
for (int i = 0; i < r; i++) {  
    for (int j = 0; j < c; j++) {  
  
        if (arr[i][j] >= 30) {  
            counter++;  
        }  
    }  
}  
  
cout << "Rooms at WARNING or above : " << counter;  
  
return 0;  
}
```

```

● PS D:\C++\Assignment\Assignment 01> cd "d:\C++\Assignment\Assignment 01\"
Floor : 1 Room : 1 : 24
Floor : 1 Room : 2 : 31.5
Floor : 1 Room : 3 : 28.0

Floor : 2 Room : 1 : 45.0
Floor : 2 Room : 2 : 22.0
Floor : 2 Room : 3 : 33.0

Floor : 3 Room : 1 : 19
Floor : 3 Room : 2 : 27.5
Floor : 3 Room : 3 : 50.2

      Room 1      Room 2      Room 3
Floor 1 :      24.0      31.5      28.0
Floor 2 :      45.0      22.0      33.0
Floor 3 :      19.0      27.5      50.2
Hottest Room : Floor 3, Room 3 -> 50.2 C
Hottest Floor : Floor 2 (avg 33.33 C)
Rooms at WARNING or above : 4
● PS D:\C++\Assignment\Assignment 01> 

```

Problem 4 — Startup Configuration via CLI

```

#include <iostream>
using namespace std;

int main(int argc, char* argv[]) {

    if (argc < 4) {
        cout << "Usage : ./sensor_monitor <warn_threshold> "
              << "<critical_threshold> <num_readings>" << endl;
        cout << "Error : Missing arguments." << endl;
        return 1;
    }

    double warn = stod(argv[1]);
    double critical = stod(argv[2]);
    int no_of_readings = stoi(argv[3]);

    if (warn >= critical) {
        cout << "Error : Warning threshold must be less than "
              << "critical threshold." << endl;
        return 1;
    }
}

```

```
if (no_of_readings < 1 || no_of_readings > 500) {  
    cout << "Error : Number of readings must be between 1 and 500."  
        << endl;  
    return 1;  
}
```

```
int a[500];
```

```
for (int i = 0; i < no_of_readings; i++) {  
    a[i] = rand() % 70;  
}
```

```
int cN = 0;  
int cW = 0;  
int cC = 0;  
int cS = 0;
```

```
for (int i = 0; i < no_of_readings; i++) {
```

```
    if (a[i] < warn) {  
        cN++;  
    }  
    else if (a[i] < critical) {  
        cW++;  
    }  
    else if (a[i] < 60) {  
        cC++;  
    }  
    else {  
        cS++;  
    }  
}
```

```
cout << "Config : Warn=" << warn << "C"  
    << " Critical=" << critical << "C"  
    << " Readings=" << no_of_readings << endl;
```

```
cout << "Results : Normal:" << cN  
    << " Warning:" << cW  
    << " Critical:" << cC  
    << " Shutdown:" << cS;
```

```
return 0;  
}
```

```
• PS D:\C++\Assignment\Assignment 01> g++ sensor_monitor.cpp -o sensor_monitor
• PS D:\C++\Assignment\Assignment 01> .\sensor_monitor 30 45 10
  Config : Warn=30C Critical=45C Readings=10
  Results : Normal:2 Warning:5 Critical:2 Shutdown:1
○ PS D:\C++\Assignment\Assignment 01> █
```

Question 2 — Embedded Memory Manager & Signal Processor

Problem 1 — The Bug in Sensor Recalibration

```
#include<iostream>
using namespace std;

// Original bug — Call by Value
void resetSensorPairV1(int reading1, int reading2){
    int temp = reading1;
    reading1 = reading2;
    reading2 = temp;
};

// Fix 1 — Call by Reference
void resetSensorPairV2(int& reading1, int& reading2){
    int temp = reading1;
    reading1 = reading2;
    reading2 = temp;
};

// Fix 2 — Call by Pointer
void resetSensorPairV3(int* reading1, int* reading2){
    int temp = *reading1;
    *reading1 = *reading2;
    *reading2 = temp;
};

int main(){

    int a = 20, b = 30;

    cout << "--- V1: Call by Value ---" << endl;
    cout << "Before   : " << "A : " << a << "    " << "B : " << b << endl;
    resetSensorPairV1(a,b);
```

```

cout << "After    : " << "A : " << a << "    " << "B : " << b << endl;

cout << "--- V2: Call by Reference ---" << endl;
cout << "Before    : " << "A : " << a << "    " << "B : " << b << endl;
resetSensorPairV2(a,b);
cout << "After     : " << "A : " << a << "    " << "B : " << b << endl;

cout << "--- V3: Call by Pointer ---" << endl;
cout << "Before     : " << "A : " << a << "    " << "B : " << b << endl;
resetSensorPairV3(&a,&b);
cout << "After      : " << "A : " << a << "    " << "B : " << b << endl;

return 0;
}

```

```

● PS D:\C++\Assignment\Assignment 01> cd "d:\C++\Assignment\Assignment 01\" ;
or } ; if ($?) { .\Bug_Sensor }
--- V1: Call by Value ---
Before    : A : 20    B : 30
After     : A : 20    B : 30
--- V2: Call by Reference ---
Before    : A : 20    B : 30
After     : A : 30    B : 20
--- V3: Call by Pointer ---
Before    : A : 30    B : 20
After     : A : 20    B : 30
○ PS D:\C++\Assignment\Assignment 01>

```

Problem 2 — Signal Processing Pipeline

```

#include <iostream>
#include <cmath>
using namespace std;

double computeRMS(double* signal, int n)
{
    double sum = 0;

    for (int i = 0; i < n; i++)
    {
        sum += (*(signal + i)) * (*(signal + i));
    }

    return sqrt(sum / n);
}

```

```
}
```

```
void normalise(double* signal, int n)
```

```
{
```

```
    double maxAbs = 0;
```

```
    // Find maximum absolute value
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        if (fabs(*(signal + i)) > maxAbs)
```

```
        {
```

```
            maxAbs = fabs(*(signal + i));
```

```
        }
```

```
    }
```

```
    // Divide every element by maxAbs
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        *(signal + i) = *(signal + i) / maxAbs;
```

```
    }
```

```
}
```

```
int countZeroCrossings(double* signal, int n)
```

```
{
```

```
    int count = 0;
```

```
    for (int i = 0; i < n - 1; i++)
```

```
    {
```

```
        if ((*(signal + i) < 0 && *(signal + i + 1) > 0) ||
```

```
            (*(signal + i) > 0 && *(signal + i + 1) < 0))
```

```
        {
```

```
            count++;
```

```
        }
```

```
    }
```

```
    return count;
```

```
}
```

```
void applyGain(double* signal, int n, double gainFactor)
```

```
{
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        *(signal + i) = *(signal + i) * gainFactor;
```

```
    }
```

```
}
```

```
int main()
{
    double signal[] = {0.5, -1.2, 0.8, -0.3, 1.0, -0.9, 0.1};

    int n = 7;
    double gainFactor = 2.0;

    cout << "Before:\n";

    for (int i = 0; i < n; i++)
    {
        cout << *(signal + i) << " ";
    }

    cout << "\n\n";

    normalise(signal, n);

    cout << "After normalise:\n";

    for (int i = 0; i < n; i++)
    {
        cout << *(signal + i) << " ";
    }

    cout << "\n\n";

    applyGain(signal, n, gainFactor);

    cout << "After applyGain:\n";

    for (int i = 0; i < n; i++)
    {
        cout << *(signal + i) << " ";
    }

    cout << "\n\n";

    cout << "RMS = " << computeRMS(signal, n) << endl;
    cout << "Zero Crossings = " << countZeroCrossings(signal, n) << endl;

    return 0;
}
```

```

● PS D:\C++\Assignment\Assignment 01> cd "d:\C++\Assignment\Assignment 01\" ;
  Processing_Pipeline } ; if ($?) { .\Processing_Pipeline }
Before:
0.5 -1.2 0.8 -0.3 1 -0.9 0.1

After normalise:
0.416667 -1 0.666667 -0.25 0.833333 -0.75 0.0833333

After applyGain:
0.833333 -2 1.33333 -0.5 1.66667 -1.5 0.166667

RMS = 1.29713
Zero Crossings = 6
○ PS D:\C++\Assignment\Assignment 01> █

```

Problem 3 — Hardware Register Access

```

#include <iostream>
using namespace std;

int main() {

    int statusReg = 0b10110001;
    int controlReg = 0b00000000;
    int dataReg = 0b11001010;

    const int* regPtr1 = &statusReg;
    cout << "Status Register : " << *regPtr1 << endl;

    regPtr1 = &dataReg;

    int* const regPtr2 = &controlReg;

    *regPtr2 = 100;
    cout << "Control Register : " << *regPtr2 << endl;

    const int* const regPtr3 = &statusReg;
    cout << "Status Register using regPtr3 : " << *regPtr3 << endl;

    return 0;
}

```

```

● PS D:\C++\Assignment\Assignment 01> cd "d:\C++\Assignment\Assignment 01\" ; if ($?) { .\HW_Reg }
Status Register : 177
Control Register : 100
Status Register using regPtr3 : 177
○ PS D:\C++\Assignment\Assignment 01> █

```

Problem 4 — Calibration Packet Parser

```

#include <iostream>
using namespace std;

```

```

bool parsePacket(const int* rawData, int size, int** outMin, int** outMax) {

```

```

    if (size <= 0) {
        return false;
    }

```

```

    *outMin = const_cast<int*>(&rawData[0]);
    *outMax = const_cast<int*>(&rawData[0]);

```

```

    for (int i = 1; i < size; i++) {

```

```

        if (rawData[i] < **outMin) {
            *outMin = const_cast<int*>(&rawData[i]);
        }

```

```

        if (rawData[i] > **outMax) {
            *outMax = const_cast<int*>(&rawData[i]);
        }
    }

```

```

    return true;
}

```

```

int main() {

```

```

    int packet[] = {45, 12, 67, 8, 55, 31};

```

```

    int* minPtr = nullptr;
    int* maxPtr = nullptr;

```

```

    if (parsePacket(packet, 6, &minPtr, &maxPtr)) {

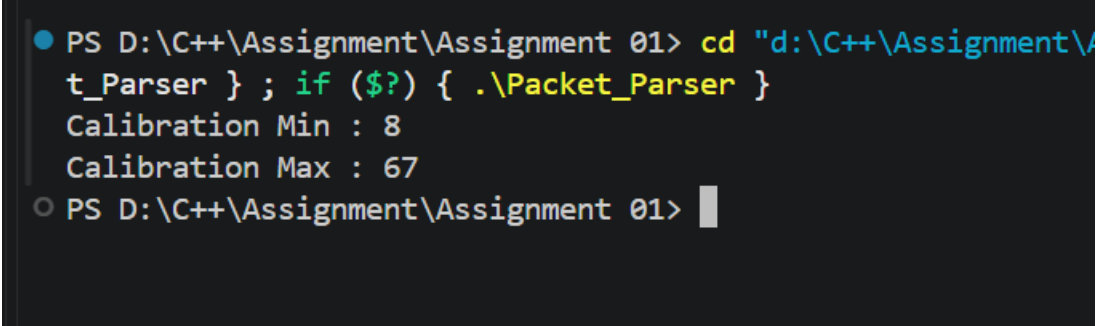
```

```

    cout << "Calibration Min : " << *minPtr << endl;
    cout << "Calibration Max : " << *maxPtr << endl;
}

return 0;
}

```



```

● PS D:\C++\Assignment\Assignment 01> cd "d:\C++\Assignment\A
t_Parser } ; if ($?) { .\Packet_Parser }
Calibration Min : 8
Calibration Max : 67
○ PS D:\C++\Assignment\Assignment 01>

```

Problem 5 — Drone Navigation Utilities

```

#include <iostream>
#include <cmath>
using namespace std;

inline double distanceBetween(double x1, double y1, double x2, double y2) {
    return sqrt(pow(x2 - x1, 2) + pow(y2 - y1, 2));
}

inline double toRadians(double degrees) {
    return degrees * (M_PI / 180.0);
}

inline double clamp(double value, double minVal, double maxVal) {
    if (value < minVal)
        return minVal;
    if (value > maxVal)
        return maxVal;
    return value;
}

inline bool isInSafeZone(double x, double y, double cx, double cy, double radius) {
    return distanceBetween(x, y, cx, cy) <= radius;
}

int main() {

```

```

double waypoints[3][2] = {
    {20.0, 30.0},
    {40.0, 40.0},
    {60.0, 10.0}
};

double homeX = 0.0, homeY = 0.0;
double radius = 50.0;

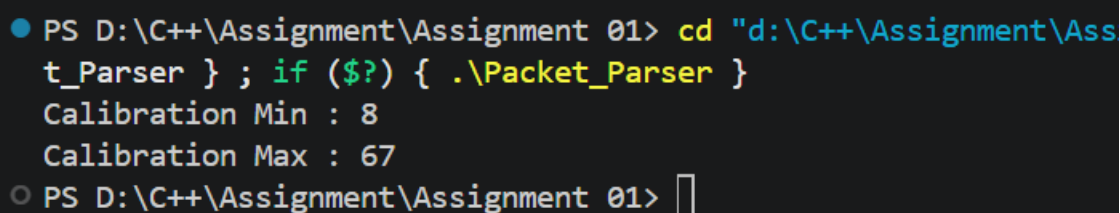
for (int i = 0; i < 3; i++) {

    double distance = distanceBetween(
        homeX, homeY, waypoints[i][0], waypoints[i][1]
    );

    cout << "Waypoint " << i + 1 << endl;
    cout << "Distance : " << distance << endl;
    cout << "Inside Safe Zone : "
        << (isInSafeZone(waypoints[i][0], waypoints[i][1],
            homeX, homeY, radius) ? "Yes" : "No")
        << endl << endl;
}

return 0;
}

```



```

● PS D:\C++\Assignment\Assignment 01> cd "d:\C++\Assignment\Ass
t_Parser } ; if ($?) { .\Packet_Parser }
Calibration Min : 8
Calibration Max : 67
○ PS D:\C++\Assignment\Assignment 01> 

```

Question 3 — HR Payroll Engine

```

#include <iostream>
#include <string>
using namespace std;

class Employee

```

```
{
private:
    int empld;
    string name;
    string department;
    char grade;
    double basicSalary;
    bool isActive;

    static int employeeCount;
    static int nextEmpld;

public:
    // Constructor
    Employee()
    {
        empld = nextEmpld++;
        employeeCount++;

        name = "";
        department = "";
        grade = 'D';
        basicSalary = 0;
        isActive = true;
    }

    // Setters
    void setName(const string& n)
    {
        if (n.empty())
        {
            cout << "ERROR: Name cannot be empty. Value rejected." << endl;
        }
        else
        {
            name = n;
        }
    }

    void setDepartment(const string& dept)
    {
        if (dept == "Engineering" ||
            dept == "HR" ||
            dept == "Finance" ||
            dept == "Operations")
```

```

    {
        department = dept;
    }
    else
    {
        cout << "ERROR: '" << dept
            << "' is not a registered department." << endl;
    }
}

void setGrade(char g)
{
    if (g == 'A' || g == 'B' || g == 'C' || g == 'D')
    {
        grade = g;
    }
    else
    {
        cout << "ERROR: Invalid grade '" << g
            << "'. Accepted values: A, B, C, D." << endl;
    }
}

void setBasicSalary(double salary)
{
    if (salary > 10000 && salary < 500000)
    {
        basicSalary = salary;
    }
    else
    {
        cout << "ERROR: Salary must be between Rs.10,000 and "
            << "Rs.5,00,000. Value rejected." << endl;
    }
}

void deactivate()
{
    isActive = false;
}

// Getters
int getEmpld() const
{
    return empld;
}

```

```
}

string getName() const
{
    return name;
}

string getDepartment() const
{
    return department;
}

char getGrade() const
{
    return grade;
}

double getBasicSalary() const
{
    return basicSalary;
}

bool getIsActive() const
{
    return isActive;
}

// Business logic
double computeAllowances() const
{
    if (grade == 'A')
        return basicSalary * 0.40;
    else if (grade == 'B')
        return basicSalary * 0.30;
    else if (grade == 'C')
        return basicSalary * 0.20;
    else
        return basicSalary * 0.10;
}

double computeGrossSalary() const
{
    return basicSalary + computeAllowances();
}
```

```

double computeTax() const
{
    double gross = computeGrossSalary();

    if (gross <= 50000)
    {
        return 0;
    }
    else if (gross <= 100000)
    {
        return (gross - 50000) * 0.10;
    }
    else
    {
        return 5000 + (gross - 100000) * 0.20;
    }
}

```

```

double computeNetSalary() const
{
    return computeGrossSalary() - computeTax();
}

```

```

void printPayslip() const
{
    cout << endl;
    cout << "===== " << endl;
    cout << "EMPLOYEE PAYSIP - AUG 2026" << endl;
    cout << "===== " << endl;

    cout << "Emp ID    : " << empId << endl;
    cout << "Name      : " << name << endl;
    cout << "Department : " << department << endl;
    cout << "Grade     : " << grade << endl;

    cout << "Status    : ";
    if (isActive)
        cout << "Active" << endl;
    else
        cout << "Inactive" << endl;

    cout << "-----" << endl;

    cout << "Basic Salary : Rs. " << basicSalary << endl;
}

```

```

cout << "Allowances ("
    << (grade == 'A' ? 40 :
        grade == 'B' ? 30 :
        grade == 'C' ? 20 : 10)
    << "%) : Rs. " << computeAllowances() << endl;

cout << "Gross Salary   : Rs. " << computeGrossSalary() << endl;

cout << "-----" << endl;

cout << "Tax Deduction   : Rs. " << computeTax() << endl;
cout << "Net Salary      : Rs. " << computeNetSalary() << endl;

cout << "===== " << endl;
}

static int getEmployeeCount()
{
    return employeeCount;
}

void acceptDetails()
{
    string inputName;
    string inputDepartment;
    char inputGrade;
    double inputSalary;

    cout << "Enter name: ";
    getline(cin >> ws, inputName);
    setName(inputName);

    cout << "Enter department: ";
    getline(cin, inputDepartment);
    setDepartment(inputDepartment);

    cout << "Enter grade: ";
    cin >> inputGrade;
    setGrade(inputGrade);

    cout << "Enter basic salary: ";
    cin >> inputSalary;
    setBasicSalary(inputSalary);
}
};

```

```

// Static member definitions
int Employee::employeeCount = 0;
int Employee::nextEmpId = 1001;

int main()
{
    // One object on stack
    Employee e1;

    // Two objects on heap
    Employee* e2 = new Employee();
    Employee* e3 = new Employee();

    e1.acceptDetails();
    e2->acceptDetails();
    e3->acceptDetails();

    /*
    e1.empId = 999;
    e1.basicSalary = -1000;

```

These produce compiler errors because empId and basicSalary are private data members. They can only be accessed through public member functions such as getters and setters.

```

*/

e1.printPayslip();
e2->printPayslip();
e3->printPayslip();

// Simulate resignation
e3->deactivate();

if (!e3->getIsActive())
{
    cout << e3->getName()
        << " is no longer active. Payroll skipped." << endl;
}

cout << "Total Employees : "
    << Employee::getEmployeeCount() << endl;

// Release heap memory
delete e2;

```

```
delete e3;

return 0;
}
```

```
=====
EMPLOYEE PAYSALIP - AUG 2026
=====
```

```
Emp ID      : 1001
Name        : Vaishali Naik
Department  : Engineering
Grade       : A
Status      : Active
-----
```

```
Basic Salary      : Rs. 490000
Allowances (40%)  : Rs. 196000
Gross Salary      : Rs. 686000
-----
```

```
Tax Deduction     : Rs. 122200
Net Salary         : Rs. 563800
=====
```

```
=====
EMPLOYEE PAYSALIP - AUG 2026
=====
```

```
Emp ID      : 1002
Name        : Priya
Department  : HR
```

=====

EMPLOYEE PAYSHEET - AUG 2026

=====

Emp ID : 1002
Name : Priya
Department : HR
Grade : B
Status : Active

Basic Salary : Rs. 250000
Allowances (30%) : Rs. 75000
Gross Salary : Rs. 325000

Tax Deduction : Rs. 50000
Net Salary : Rs. 275000

=====

=====

EMPLOYEE PAYSHEET - AUG 2026

=====

Emp ID : 1003
Name : Krystal
Department :

=====

EMPLOYEE PAYSHEET - AUG 2026

=====

Emp ID : 1003
Name : Krystal
Department :
Grade : C
Status : Active

Basic Salary : Rs. 45000
Allowances (20%) : Rs. 9000
Gross Salary : Rs. 54000

Tax Deduction : Rs. 400
Net Salary : Rs. 53600

=====

Krystal is no longer active. Payroll skipped.
Total Employees : 3

PS D:\C++\Assignment\Assignment 01> █